Film & Kino Agent

Sitzung 01 – Kick-Off

Prof. Dr. Richard Sieg
SoSe 2026

DIS22a

Agenda

01. Organisatorisches

02. Das Thema

03. Tutorial

04. Erster Meilenstein

05. Set-Up

Ziele des Seminars

Eine Einführung in Implementationen von

Large Language Models

und Agenten

in ein Produkt

mit eigenen Daten

anhand eines klaren & konkreten Beispiels

01.

Organisatorisches

Über mich

- Professor für NLP & LLMs seit 1.12.2025 am IWS
- 2017 – 2025 NLP & ML Engineer in Kölner Start-Ups
- Promotion: Diskrete Mathematik (Universität Osnabrück)
- ~17 Jahre Erfahrung in Hochschullehre
  - Seit 2023 Lehrbeauftragter am IWS für Projektseminare
- richardsieg.github.io

Was war euer letzter Film im Kino?

Termine

Datum	Inhalt
15.4	Kick-Off, Projektvorstellung, Setup
22.4	UI bauen, LLM-API-Anbindung, Prompt Engineering
29.4	Vektordatenbanken & Embeddings
6.5	Filmdatenbank vektorisieren
13.5	*Home Office*
20.5	RAG: Theorie + Implementation (Projektwoche)
27.5	*Home Office*
3.6	Tool Use & Agenten
10.6	Agenten-Implementation + Theorie: Evaluation
17.6	Gegenseitiger Nutzertest + Feinschliff
24.6	Feinschliff, Bugfixes
1.7	Abschlusspräsentationen

Exam Registration Phase

The exam registration phase is from 20.04.2026 until 04.05.2026

Anforderungen

- Gut dokumentierter und funktionsfähiger Code
- Insbesondere funktionierende Implementationen der Meilensteine
- Innerhalb des Codes: Kurze Erläuterungen zu den Ergebnissen
- Mindestens ein neues LLM Tool, das im Seminar nicht besprochen wurde

Abschlusspräsentation

- Mündliche Vorstellung des Codes von jedem einzelnen Gruppenmitglied (ohne Folien)
- Ca 10 Minuten pro Mitglied
- Selbstständige Aufteilung innerhalb des Teams
- Nach jeder Vorstellung kleine Fragerunde
- Termin: 01.07.2026

Praxispartner

koeln.de

Home » Kinoprogramm für Köln

Kinoprogramm: Alle Filme in Köln

Top-Filme | Filme A-Z | Heute | Alle Tage | Kinos

Der Astronaut - Project Hail Mary

Ryland Grace erwacht an Bord eines Raumschiffs und kann sich an nichts mehr erinnern. Als er beginnt, seine Umgebung zu erkunden, kehrt sein Gedächtnis langsam zurück. Auf der Erde war er Naturwissenschaftler ...

Land	USA
Länge	156 Minuten
Jahr	2026
FSK	12
Regie	Christopher Miller, Phil Lord
Darsteller	Ryan Gosling, Sandra Hüller, Milana Vayntrub
Genre	Science Fiction

Spielzeiten

Marty Supreme

Marty Mauser arbeitet als Schuhverkäufer in New York und träumt davon, der beste Tischtennisspieler der Welt zu werden. Ein großes Turnier in Europa soll ihn seinem Traum ein Stück näherbringen. Doch im Finale ...

koeln.de wird eure Agenten Mitte Juni testen und Feedback geben

Teams

02.

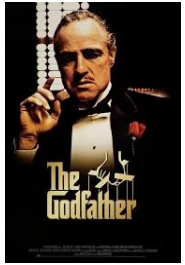
Das Thema

Übersicht unseres Kurses

- UI mit Streamlit - Beliebte Plattform zum Bauen von (ML) Prototypen
- LLMs - Verwendung von LLMs in Python
- Vektordatenbanken - Vector Embeddings durch LLMs, Ähnlichkeit von Vektoren, Such- und Indexalgorithmen, Implementation mit qdrant & chromadb
- RAG - Prinzip & Implementation
- Agenten & Tool Use – Anbindung von LLMs an Funktionen, wie das aktuelle Kölner Kinoprogramm; Agent entscheidet selbstständig über das Tool
- Evaluation von LLM Anwendungen

Ich suche Mafiafilme.

Klar, hier sind einige Mafiafilme.



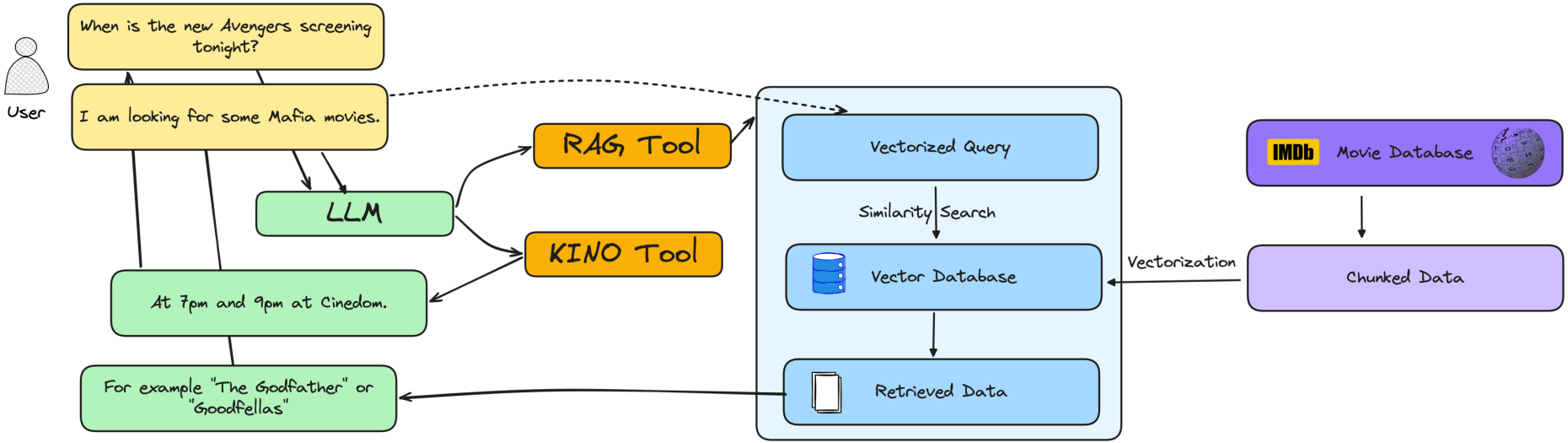
Wer ist der Hauptdarsteller in Goodfellas?

Der Hauptdarsteller in "Goodfellas" ist Ray Liotta. Er spielt Henry Hill, einen ehemaligen Gangster.

Laufen gerade Mafiafilme in Köln?

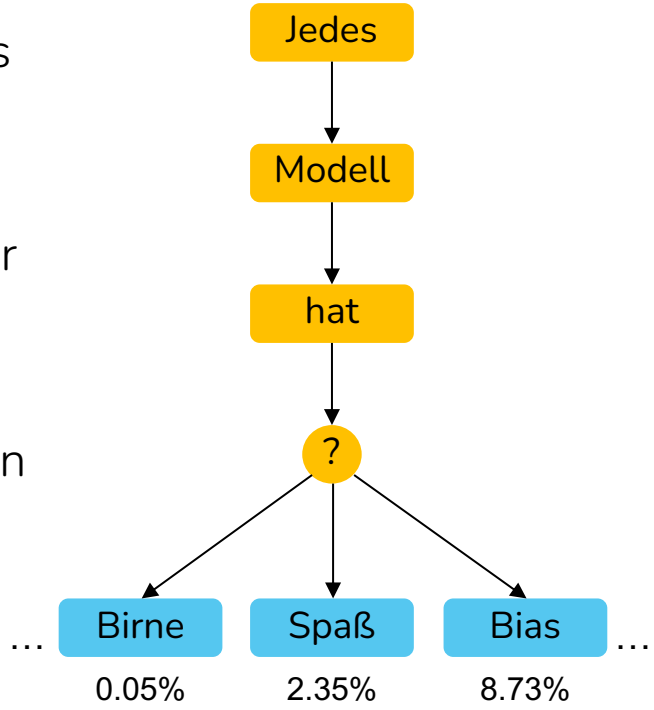
Ja gerade läuft tatsächlich Scarface im Odeon.

Architektur

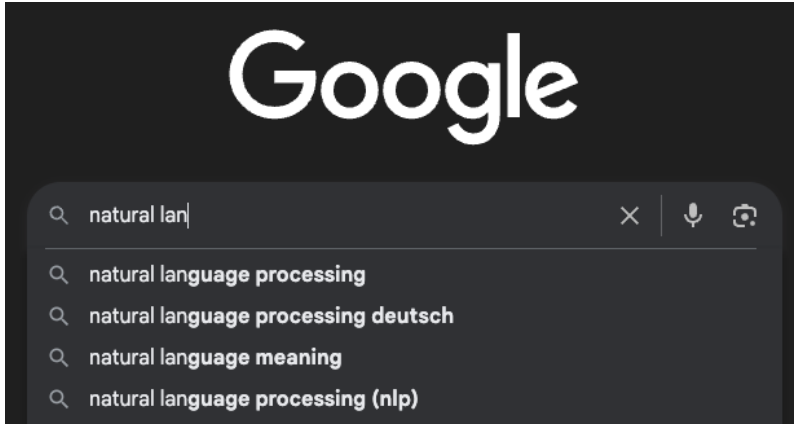


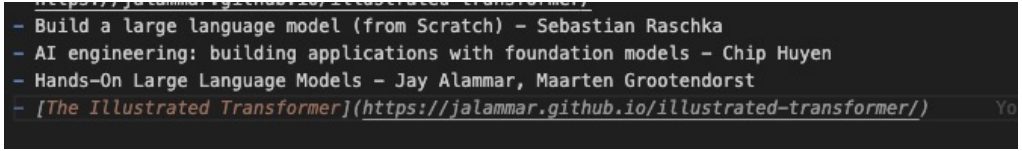
Was ist ein Sprachmodell?

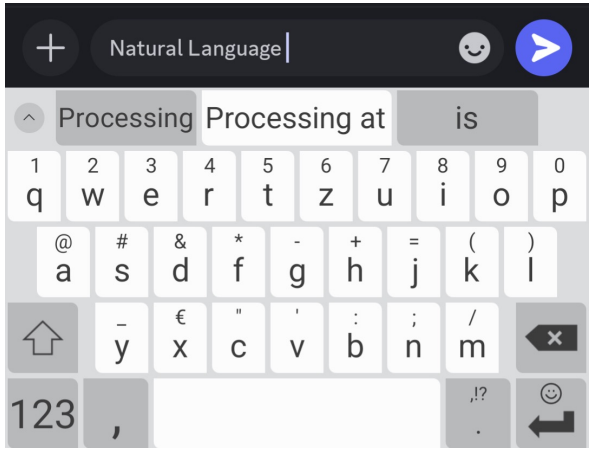
- Ein statistisches Modell, das basierend auf einer Sequenz an *"Wörtern"* Wahrscheinlichkeiten für das nächste Wort berechnet
- Basieren fast immer auf ***neuronalen Netzen*** und dabei inspesondere auf der ***Transformer*** Architektur aus 2017 (Grund siehe später)
  GPT: Generative Pretrained Transformer
- Sprachmodelle ***trainieren*** auf riesigen Datenmengen
- Trainingsziel meistens: das nächste korrekte Wort vorherzusagen
- ***Mehr Details in unserer NLP Vorlesung***



Sprachmodelle sind überall







Historie der Sprachmodelle

~1950s-1990s

Statistical LMs

N-grams and Markov models

~Early 2000s

Feedforward Neural LMs

Neural networks, Learned word embeddings jointly.

2013

Word2Vec

Efficient dense word embeddings enable new applications

~2014-2017

RNNs/LSTMs

Recurrent networks improve sequence handling

2017

Transformer

Attention mechanism revolutionizes language modeling

2018-Present

Large Language Models

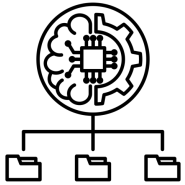
BERT, GPT, T5, etc.

Exkurs: Tokens

- Computer verstehen ”nur Zahlen”, also brauchen wir eine Indexierung (Wörter → Index)
- Alle Wörter durch zu nummerieren, führt zu zu vielen Indizes und wir verlieren die Information von Teilwörtern oder Silben (“Bibliothek” und “bibliothekarisch”)
- Moderne Tokenizer sind speziell trainierte Algorithmen, die besonders häufige Buchstabenkombinationen zusammen führen
  - Der Eingabetext wird dann zunächst in Tokens umgewandelt
  - Fixe Menge an möglichen Tokens (~100k)
  - Meist über mehrere Sprachen und Alphabete hinweg

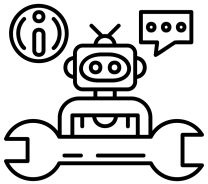
GPT Tokenizer Playground

Der Sprung zu LLMs



Skalierung

- Die Transformer Architektur erlaubt es das Training enorm zu skalieren (↗️ Trainingsdaten, ↗️ Compute Power, ↗️ Modellgröße)
- Parallelisierung auf Rechenclustern
- Emergenz von nicht trainierten Fähigkeiten



Menschliche Präferenz

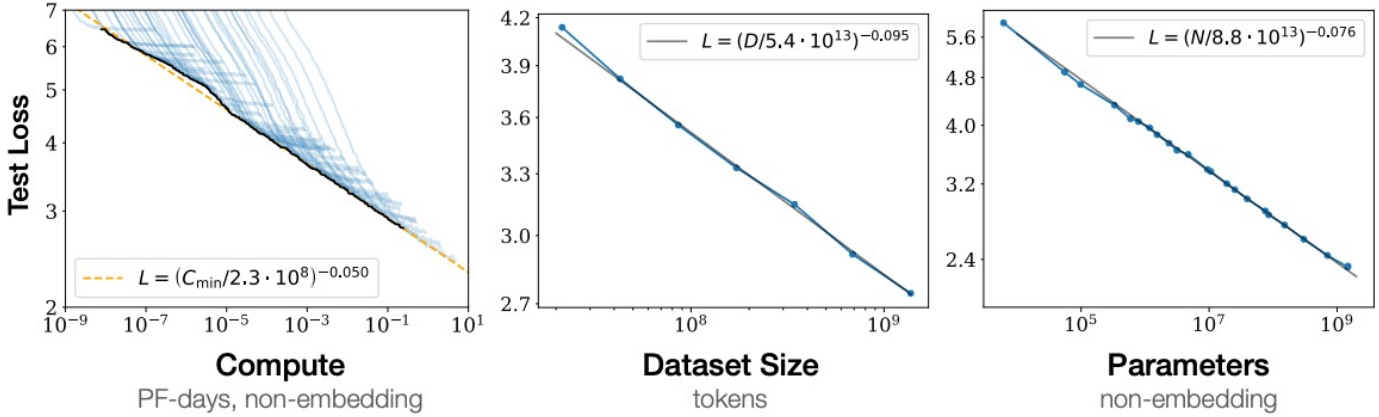
- Nach dem Pre-Training findet ein Fine-Tuning statt, um LLMs dazu zu bringen, von Menschen präferierte Antworten zu generieren

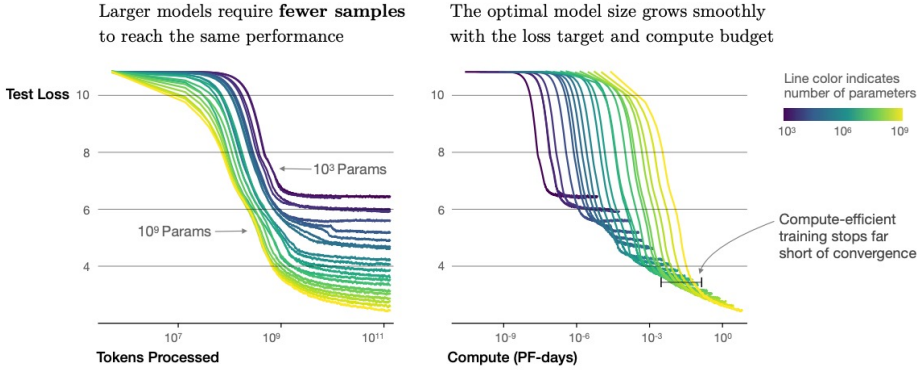
Prompt: Können LLMs Bias haben?

1 Jedes LLM hat Bias.

2 Nur manche. GPT nie!

Der Sprung zu LLMs

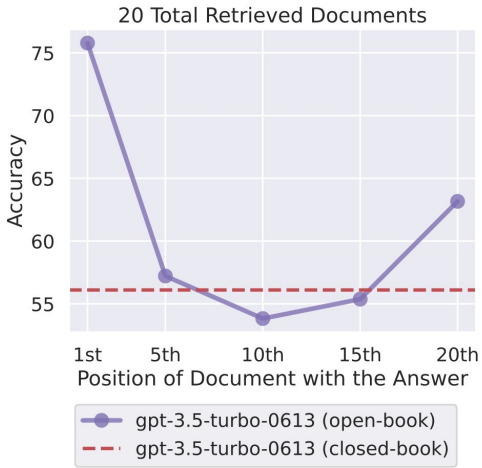




Context Window

- LLMs können nur eine begrenzte Anzahl von Tokens gleichzeit verarbeiten
- Vor allem in den Anfangstagen war das ein Problem (nur um die Tausend)
- Nun gibt es viele Modelle mit 1M Tokens, unter 100k eigentlich keine
- Trotzdem Probleme mit langem Kontext (*Needle in a Haystack*, *Lost in the Middle*)
- Techniken für Verarbeitung von viel Information nächste Sitzung (RAG)
- Forschungs- und Entwicklungsherausorderung:

Den LLMs den relevanten Kontext in der passenden Form geben.



Parameter & System Prompt

- Es gibt viele Parameter, die die Textgenerierung von LLMs steuern
  - ***Temperature:*** Richtung 0: ”deterministischer”, Richtung 1: “zufälliger/kreativer”
- ***System Prompt***
  - Die Rollenbeschreibung für ein bestimmte Szenario
  - Was sind die Aufgaben des LLMs? Welche Regeln soll es befolgen? Stil?
  - Tips & Tricks nächste Sitzung

> You are a movie recommondation chatbot.
>
> You have access to a RAG and a Cologne theater tool.
>
> Always answer nicely.

Entwicklung: Reasoning Modelle

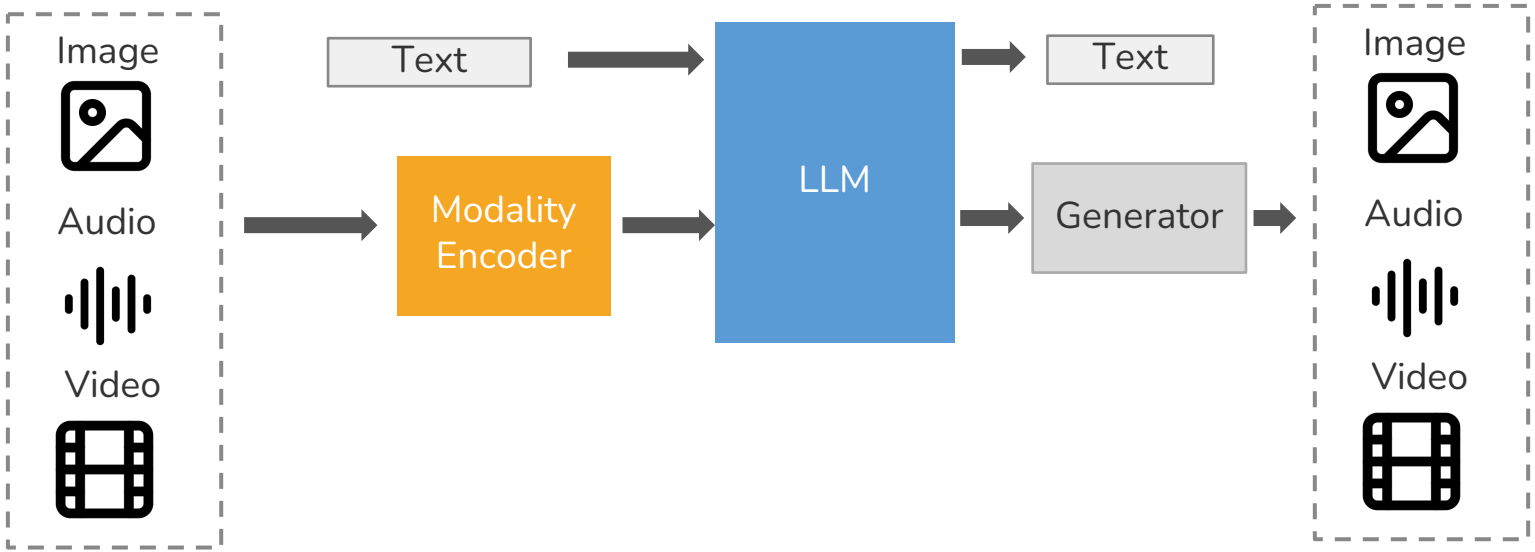
- Reasoning Modelle geben nicht direct eine Antwort, sondern generieren erst Tokens, um über das Problem “nachzudenken” und sind speziell auf Schritt für Schritt Überlegungen trainiert
  - (Maximale) Anzahl Reasoning Tokens können meist konfiguriert werden
- Meist für komplexere, Schritt für Schritt Aufgaben sinnvoll
- Erstes Reasoning Modell OpenAI o1, seit 2025 hat fast jedes Modell reasoning
- ChatGPT 5 entscheidet auf der UI selbst, ob es reasoning benutzt oder nicht

Entwicklung: Websuche & Deep Research

- Seit 2024: Websuche als mögliches ***Tool***, welches das LLM nutzen kann mit der Verlinkung von Quellen
- Seit Feb2025 Deep Research: Internetrecherche über mehrere Minuten und Erstellung eines Berichts
- Wir werden im Laufe des Seminars unsere eigenen Tools schreiben

Entwicklung: Multimodale Modelle

- Medien werden für ein LLM “verständlich” encodiert
- Erzeugung von Medien oftmals über den Aufruf von eigens trainierten Modellen
- Vor allem nützlich zur Dokumentenanalyse



LLM als Chat ⟷ LLM im Produkt

- ChatGPT war das erste Produkt, welches um ein LLM herum gebaut wurde
- Chat ist das natürliche Habitat für ein allgemeines LLM
- Durch verschiedene System Prompts, Konnektoren und Dateizugriff lassen sich Chat Interfaces in gewissem Grad zu rudimentären Anwendungen modellieren
- Diametral dazu sind Aufrufe von LLMs innerhalb eines Programmcodes und Verarbeitung der Antworten
- Dazwischen liegen Workflow Builder wie n8n, flowise und Langflow

einfach ⟷ komplex

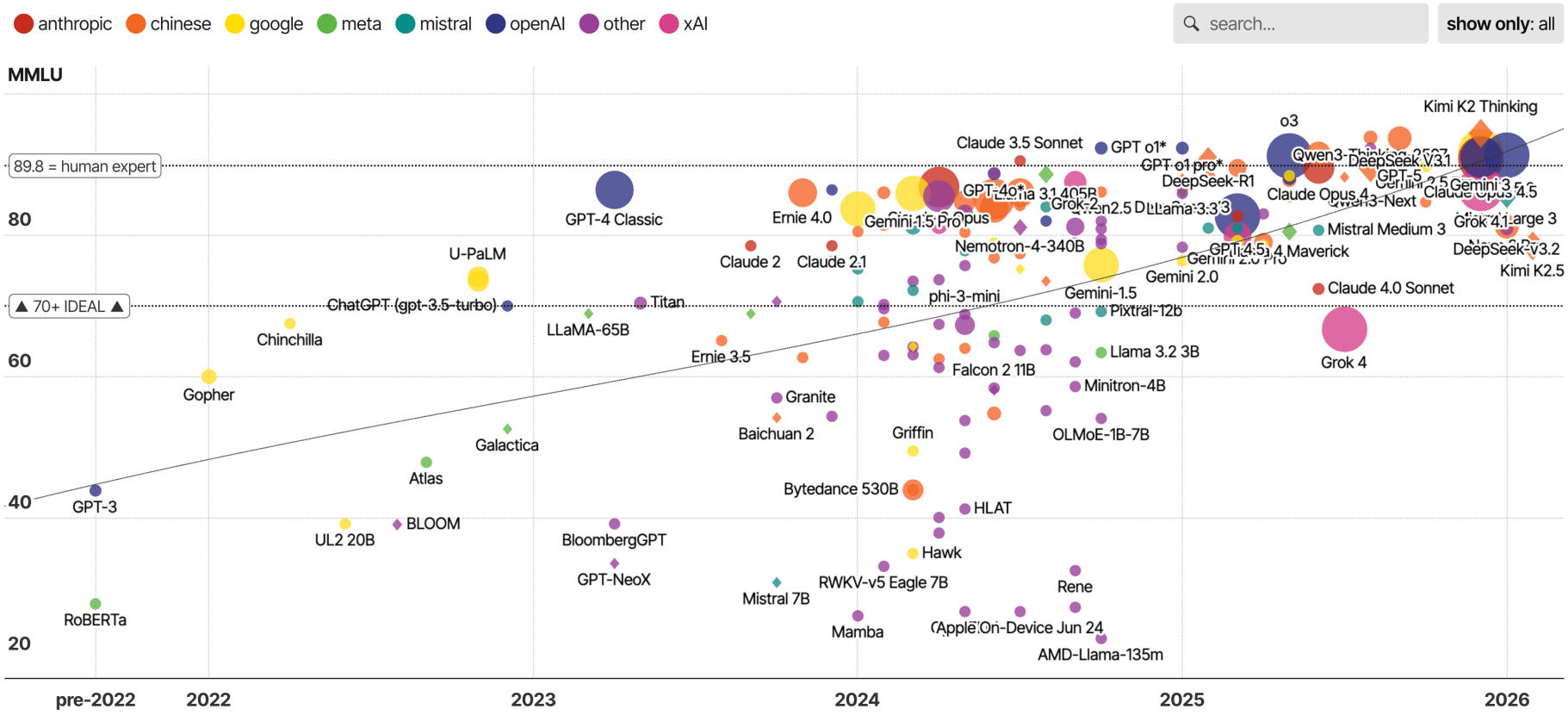
starr ⟷ flexibel

```python
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5.4",
    input="Write a short bedtime story about a unicorn."
)
```

LLMs programmatisch Nutzen

- Einbindung der LLMs in den Quellcode durch entsprechende Bibliotheken oder direkte API Aufrufe
- Die Antworten werden dann im Code weiter verarbeitet
- Die meisten LLMs bieten die Möglichkeiten Antworten in einem bestimmten Format zu erzwingen → bessere Verarbeitung im Code möglich

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5.4",
    input="Write a short bedtime story about a unicorn."
)
```



Quelle: Information is beautiful

GenAI Landschaft

- Drei große ***closed source*** US Player: Anthropic (Claude), Google (Gemini), OpenAI (GPT)
- Open Source: Trainingsdaten, Algorithmen und Gewichte sind verfügbar
  - OLMo (AI2, Seattle)
  - T5 (Google)
  - Apertus (Swiss AI, ETH Zürich)
- Open Weight (fast alle anderen): nur die Gewichte sind verfügbar
- Chinesische Open Weight Modelle: DeepSeek, Qwen, Kimi
- Weitere Open Weight: Llama (Meta), GPT OSS, Mistral, Gemma (Google), Phi

LLMs aus Europa

- Bislang ist Mistral das einzige wettbewerbsfähige Unternehmen und LLM
  - Neueste Version: Mistral 3 und Mistral 3 Large (Dez. 2025)
- Kleinere Initiativen aus Deutschland und der Schweiz
- Es gab/gibt einen Anbieter aus Heidelberg: Aleph Alpha

Braucht die deutsche Vorzeige-KI mehr Erziehung?

Die KI der deutschen Firma Aleph Alpha gilt als vielversprechendstes Produkt Europas. Doch sie generiert rassistische Texte. Das könnte zum Problem in Anwendungen werden.

- Neue Servercluster werden gerade aufgebaut (z. B. OpenEuroLLM inkl. Jülich)
- Ist das sinnvoll – und vielleicht sogar notwendig?

03.

Tutorial

04.

Erster Meilenstein

Erster Meilenstein

1. Gebt eurer Applikation einen griffigen Namen und setzt ihn als Header in Streamlit.
2. Legt ein eigenes Streamlit Theme an. Macht euch auch gerne eigene Icons für die Avatare. Und wenn ihr wollt auch gerne einen Dark-Light Mode.
3. Fügt ein neues Menü “All Movies” in der Navigation hinzu. Lest in dem Menü die `data/tmdb_5000_movies.csv` Datei von Filmen ein.
   - Option A: Zeigt die Titelbilder der Filme mit einem 4 Column Design an mit Titel als Caption. Der Datensatz an sich hat noch nicht die Poster. Die kann man aber über die TMDB API bekommen, die wir eh noch brauchen werden.
   - Option B: Die Filme als Liste anzeigen, mit den Zusatzinfos aus der der csv.

   - Optional: Die Titelbilder oder die Listeneinträge funktionieren als Link, um auf die IMDb Seite des Films zu kommen.
4. Bindet Gemini Flash 2.5 als mögliches LLM ein.
5. Bindet Mistral 3.2 Small als mögliches LLM ein.

05.

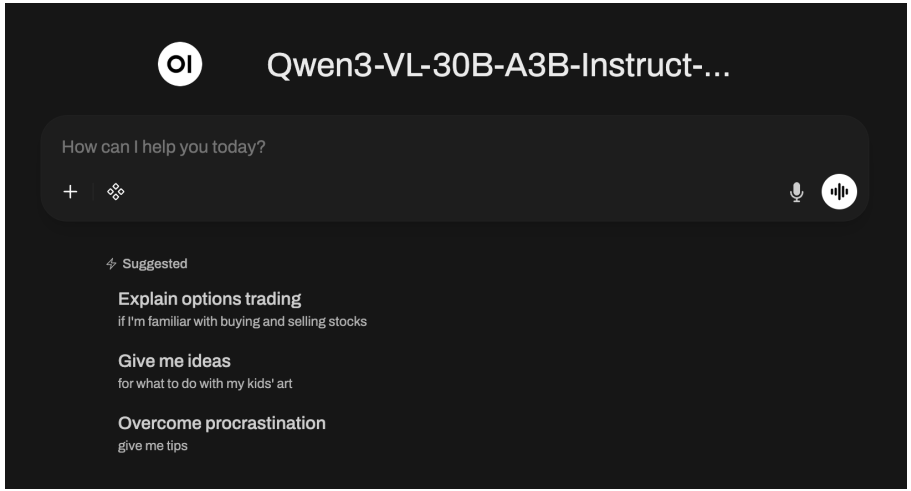
Set-Up

Projektstruktur

- Empfehlung: VS Code als IDE
  - marimo extension oder direkt im Browser
  - Start des Chatbots über ein justfile (aber nicht zwingend)
- Ein GitHub Repository pro Team
- Aufgaben, Diskussionen und Fragen über Git Issues
  - @RichardSiegTH mention für Notifications
- Python Pakete über virtual environments und uv

OpenWebUI

- Open Source Alternative für Chat und API Anbindung an LLMs
- Läuft auf einem Cluster in Mainz unter iwschat.service.kitegg.hs-mainz.de
- Passwort solltet ihr per Email erhalten haben
- Unter Nutzereinstellungen könnt ihr euch einen API Key anlegen



GitHub

- Ihr macht einen Fork von unserem Hauptrepository
- Dann macht ihr einen initialen Clone von eurem Repo.
- Ihr arbeitet immer auf dem “main” Branch. Ich gebe euch Feedback über einen Merge Request auf den Feedback Branch.
- Als Student*innen bekommt ihr GitHub Education und Copilot umsonst

uv

- Neuer Package Manager, der alle anderen abgelöst hat (pip, poetry etc) und sehr schnell ist
- Verwaltung der Pakete in einer **`pyproject.toml`**
- Python Installation: **`uv python install 3.13`**
- Erste Installation und venv erzeugen: **`uv sync`**
- Skripte ausführen: **`uv run …`**
  - **`uv run streamlit src/chatbot.py`**
- Paket hinzufügen: **`uv add …`**

marimo

- Neue Alternative zu Jupyter Notebooks
- Liegt in Python Dateien (-> bessere Diffs & Imports)
- Reaktiv: Abhängige Zellen werden automatisch ausgeführt
- Interaktiv: Viele mögliche Input Optionen
- Coding Assistant über GitHub Copilot & LLM APIs (OpenAI etc)
- Editor: Im Browser oder VSCode/Cursor Erweiterung
- Editor starten: `marimo edit`
- Als Web App ausführen: `marimo run notebook.py`

Set-Up

1. Fork vom Repo und Name ändern
2. uv installieren
3. uv sync
4. Google Cloud Vertex AI – Gen AI Studio API Key beschaffen und in die .env Datei
5. OpenWebUI API erzeugen und in die .env Datei
6. marimo starten und schauen, ob alles funktioniert
7. Chatbot starten